

ROS 2 Foundations

DR. TNOURJI, ABDELLAH

Research Scientist

The UM6P Vanguard Center

Autonomous Drone Engineering Internship

Mohammed VI Polytechnic University, UM6P , September 15, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

1 ROS 2 Humble Foundations

1 ROS 2 Humble Foundations

- Environment and First Observation
- The ROS 2 Graph and Nodes
- Topics, Publishers, Subscribers, and Messages
- Services and Actions
- Parameters, Packages, and Workspaces
- Python Publisher and Subscriber
- Drone-System Integration
- Troubleshooting and Practice

From Python to ROS 2

Python provides logic; ROS 2 connects that logic to the robotic system

A drone is not one program. It is a team of communicating components.

From Python to ROS 2

Python provides logic; ROS 2 connects that logic to the robotic system

A drone is not one program. It is a team of communicating components.

Perception

Camera acquisition and object detection.

Mission Logic

Trajectory generation and inspection decisions.

Flight Interface

Telemetry reception and
PX4 setpoint publication.

ROS 2 Role

ROS 2 provides the communication and integration framework that allows these components to discover one another and exchange typed information.

What Is ROS 2?

A modular software framework for building robot systems

Intuitive Definition

ROS 2 is a collection of software libraries, command-line tools, conventions, and communication mechanisms used to create and integrate robotic applications.

What Is ROS 2?

A modular software framework for building robot systems

Intuitive Definition

ROS 2 is a collection of software libraries, command-line tools, conventions, and communication mechanisms used to create and integrate robotic applications.

Team Analogy

Imagine a technical team in which each specialist has one responsibility. ROS 2 provides the shared language, communication channels, addresses, and rules that let the specialists cooperate.

Important Distinction

ROS 2 is not the computer's operating system. Ubuntu manages hardware, files, memory, users, and processes. ROS 2 runs on top of Ubuntu and organizes communication between robotics software components.

Why Use ROS 2?

ROS 2 helps separate programs work together

A complete robot is not one large program.

It is a team of smaller programs with different responsibilities.

Why Use ROS 2?

ROS 2 helps separate programs work together

A complete robot is not one large program.

It is a team of smaller programs with different responsibilities.

Separate

Each component performs one clear task: camera, mission, control, perception, or logging.

Connect

ROS 2 allows the components to exchange structured messages through defined interfaces.

Observe

ROS 2 tools help engineers inspect which components are running and what information they exchange.

Why Use ROS 2?

ROS 2 helps separate programs work together

A complete robot is not one large program.

It is a team of smaller programs with different responsibilities.

Separate

Each component performs one clear task: camera, mission, control, perception, or logging.

Connect

ROS 2 allows the components to exchange structured messages through defined interfaces.

Observe

ROS 2 tools help engineers inspect which components are running and what information they exchange.

Drone Project Example

The camera produces images → the detector identifies objects → the mission node uses the observations → the results are visualized and recorded.

Central Idea

ROS 2 does not fly the drone and does not detect objects. It is the communication and integration framework that helps the specialized software components work together.

Course Compatibility Contract

Every command and exercise targets one supported baseline

Ubuntu 22.04 LTS + ROS 2 Humble Hawksbill

Student Rule

Use the versions specified by the internship repository. Do not silently replace Humble commands or packages with instructions written for another ROS 2 distribution.

Why This Matters

Package names, supported platforms, simulator combinations, APIs, and setup instructions may differ between ROS 2 distributions. Newer does not automatically mean compatible with this project.

1 ROS 2 Humble Foundations

- Environment and First Observation
 - The ROS 2 Graph and Nodes
 - Topics, Publishers, Subscribers, and Messages
 - Services and Actions
 - Parameters, Packages, and Workspaces
 - Python Publisher and Subscriber
 - Drone-System Integration
 - Troubleshooting and Practice

Live Check 1: Load ROS 2 Humble

Each new terminal must receive the ROS 2 environment

Execute

```
source /opt/ros/humble/setup.bash
printenv ROS_DISTRO
which ros2
ros2 --help
```

Expected Evidence

ROS_DISTRO should report humble, and which ros2 should locate the ROS 2 command-line executable.

Interpretation

source changes the current shell environment. It does not reinstall ROS 2. Repeat it in every new terminal unless the project defines an approved automatic setup.

Live Check 2: Diagnose the Environment

Verify the installation before blaming application code

Execute

```
ros2 doctor --report
```

What to Inspect

Confirm the ROS 2 distribution, platform, middleware information, network interfaces, and package environment. Read warnings in context rather than treating every line as a fatal error.

Engineering Habit

Preserve the command and complete output when asking for help. A screenshot of only the last line may hide the actual cause.

Live Demonstration: Start Turtlesim

Use a small system before connecting the drone stack

Terminal 1

```
source /opt/ros/humble/setup.bash  
ros2 run turtlesim turtlesim_node
```

Terminal 2

```
source /opt/ros/humble/setup.bash  
ros2 run turtlesim turtle_teleop_key
```

What to Observe

Two executables run as separate processes. One displays the simulated turtle; the other converts keyboard input into movement commands.

Terminal Focus

Keep Terminal 2 active while pressing arrow keys. Stop each foreground process deliberately with `Ctrl+C`.

1 ROS 2 Humble Foundations

- Environment and First Observation
- **The ROS 2 Graph and Nodes**
- Topics, Publishers, Subscribers, and Messages
- Services and Actions
- Parameters, Packages, and Workspaces
- Python Publisher and Subscriber
- Drone-System Integration
- Troubleshooting and Practice

A live map of components and communication relationships

Nodes + topics + services + actions + parameters = **ROS 2 graph**

Intuition

The graph is not a file stored once. It is the changing network of ROS 2 entities that currently exist and communicate while the system runs.

Drone Example

An orbit controller, PX4 bridge, image bridge, detector, and evidence logger appear as nodes connected through named interfaces.

A Simple ROS 2 Computation Graph

How a mobile robot exchanges sensor data and motion commands



Sensor Information

The robot driver acts as a **publisher** on:

`/scan`

It publishes laser measurements using:

`sensor_msgs/msg/LaserScan`

Any node interested in obstacle detection can **subscribe** to this topic.

Motion Commands

The robot driver acts as a **subscriber** on:

`/cmd_vel`

It receives velocity commands using:

`geometry_msgs/msg/Twist`

A controller publishes the desired linear and angular velocities on this topic.

A Simple ROS 2 Computation Graph

How a mobile robot exchanges sensor data and motion commands



Sensor Information

The robot driver acts as a **publisher** on:

`/scan`

It publishes laser measurements using:

`sensor_msgs/msg/LaserScan`

Any node interested in obstacle detection can **subscribe** to this topic.

Motion Commands

The robot driver acts as a **subscriber** on:

`/cmd_vel`

It receives velocity commands using:

`geometry_msgs/msg/Twist`

A controller publishes the desired linear and angular velocities on this topic.

Key Intuition

The robot driver connects ROS 2 to the robot or simulator. It publishes what the robot senses and subscribes to what the software wants the robot to do.

ROS 2 Names and Namespaces

Organizing communication in single-robot and multi-robot systems

Names Work Like Filesystem Paths

ROS 2 resources use structured names beginning with /. A namespace adds a common prefix that groups related nodes and topics.

Single Robot

`/cmd_vel`

`/scan`

Two Robots

`/drone_1/cmd_vel`

`/drone_1/scan`

`/drone_2/cmd_vel`

`/drone_2/scan`

ROS 2 Names and Namespaces

Organizing communication in single-robot and multi-robot systems

Names Work Like Filesystem Paths

ROS 2 resources use structured names beginning with /. A namespace adds a common prefix that groups related nodes and topics.

Single Robot

`/cmd_vel`

`/scan`

Two Robots

`/drone_1/cmd_vel`

`/drone_1/scan`

`/drone_2/cmd_vel`

`/drone_2/scan`

Why Namespaces Matter

Both robots may use the same node design and message types, while their namespaces keep their commands and sensor data separate.

Drone Project Connection

A command published to `/drone_1/cmd_vel` should not accidentally control `drone_2`.

The ROS 2 Graph

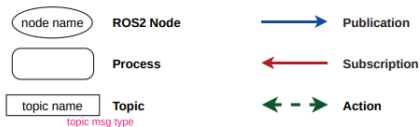


Figure 1.3: Description of symbols used in computer graph diagrams.

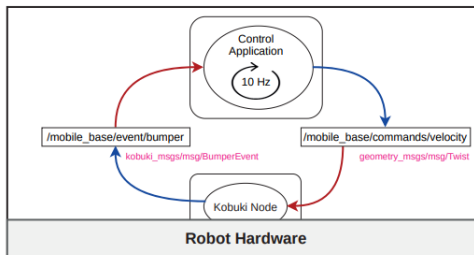


Figure 1.4: Computing graph of a simple control for the Kobuki robot. The control application publishes speeds computed from the information of the bumper to which it subscribes.

[5]

What Is a ROS 2 Node?

One software specialist responsible for one part of the robot

A robotic system is a team of specialized programs.

In ROS 2, each running specialist is called a **node**.

What Is a ROS 2 Node?

One software specialist responsible for one part of the robot

A robotic system is a team of specialized programs.

In ROS 2, each running specialist is called a **node**.

Receive

A node can receive information from other nodes.

Example: receive a camera image.

Process

A node performs one focused software task.

Example: detect an object.

Send

A node can share its result with other nodes.

Example: publish detections.

What Is a ROS 2 Node?

One software specialist responsible for one part of the robot

A robotic system is a team of specialized programs.

In ROS 2, each running specialist is called a **node**.

Receive

A node can receive information from other nodes.

Example: receive a camera image.

Process

A node performs one focused software task.

Example: detect an object.

Send

A node can share its result with other nodes.

Example: publish detections.

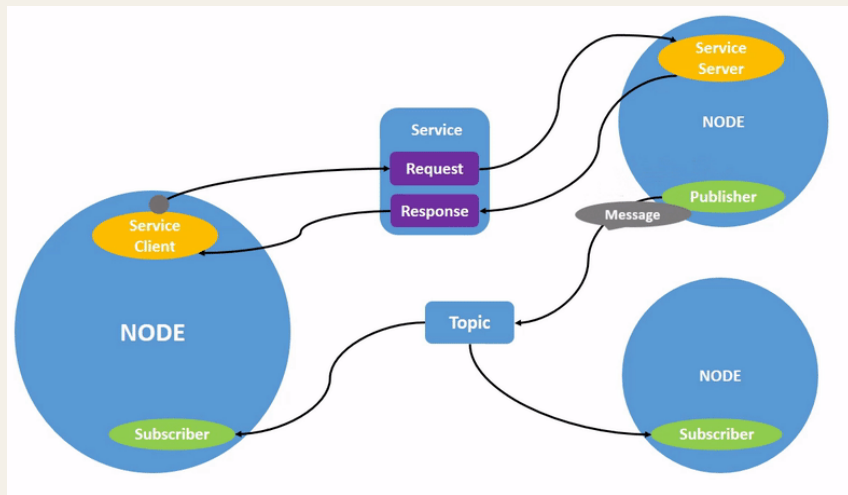
Drone Project Example

The **camera node** publishes images → the **detector node** identifies objects → the **mission node** uses the observations → the **logger node** records evidence.

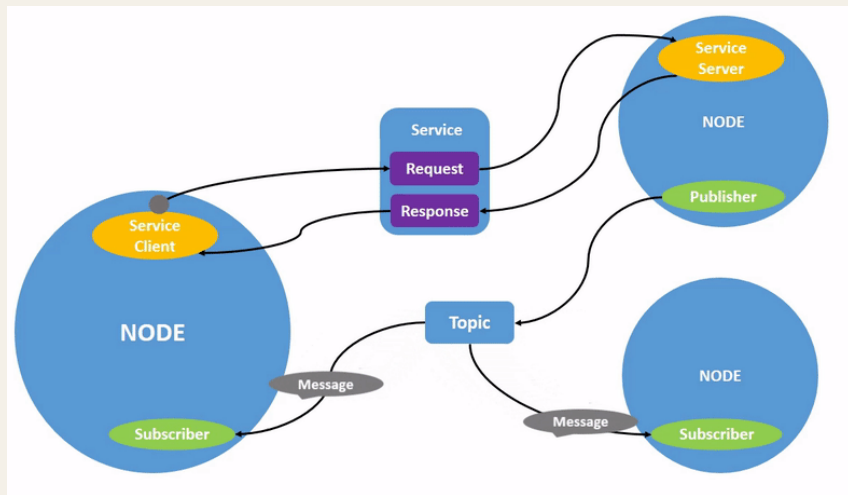
Central Idea

A node is not a physical sensor or part of the drone. It is a running software component with one clear responsibility inside the complete robotic system.

ROS 2 Demonstration



ROS 2 Demonstration



Live Tutorial: Discover Nodes

Ask the running system who is participating

Keep Turtlesim Running, Then Execute

```
ros2 node list  
ros2 node info /turtlesim  
ros2 node info /teleop_turtle
```

Read the Output

`ros2 node list` identifies active node names. `ros2 node info` reveals publishers, subscribers, services, actions, and interfaces associated with one node.

Project Transfer

Later, use `ros2 node list` to verify that the orbit controller, bridges, and perception nodes actually started. A terminal window alone is not sufficient evidence.

Reading `ros2 node info`

Do not read every line first; identify the node's communication roles

The Main Question

What information does the `/turtlesim` node **receive**, **produce**, and **provide on request**?

Reading ros2 node info

Do not read every line first; identify the node's communication roles

The Main Question

What information does the `/turtlesim` node **receive**, **produce**, and **provide on request**?

Receives

Subscribers

`/turtle1/cmd_vel`

The node receives movement commands as Twist messages.

Produces

Publishers

`/turtle1/pose`

`/turtle1/color_sensor`

The node publishes its position and sensor information.

Provides

Services and Actions

`/spawn`

`/reset`

`/rotate_absolute`

Other nodes can request operations or send a goal.

Reading ros2 node info

Do not read every line first; identify the node's communication roles

The Main Question

What information does the `/turtlesim` node **receive**, **produce**, and **provide on request**?

Receives

Subscribers

`/turtle1/cmd_vel`

The node receives movement commands as Twist messages.

Produces

Publishers

`/turtle1/pose`

`/turtle1/color_sensor`

The node publishes its position and sensor information.

Provides

Services and Actions

`/spawn`

`/reset`

`/rotate_absolute`

Other nodes can request operations or send a goal.

Receive commands → update the simulation → publish observations

Do Not Be Overwhelmed

Entries containing `parameter` and `/rosout` are normal ROS 2 support interfaces. At the beginning, focus only on the interfaces directly related to the turtle's motion and observations.

1 ROS 2 Humble Foundations

- Environment and First Observation
- The ROS 2 Graph and Nodes
- Topics, Publishers, Subscribers, and Messages
- Services and Actions
- Parameters, Packages, and Workspaces
- Python Publisher and Subscriber
- Drone-System Integration
- Troubleshooting and Practice

Topics: Continuous Information Channels

Publishers send data; subscribers receive matching data

Publisher → named topic → **Subscriber**

Intuition

A topic resembles a radio channel. Publishers transmit without addressing one specific receiver. Any compatible subscriber may listen to the named channel.

Drone Examples

Camera images, vehicle odometry, desired paths, detection results, and telemetry are natural topic data because they may be produced repeatedly over time.

Publishers and Subscribers

Communication roles are independent from node names

Publisher

Creates messages of a defined type and sends them on a named topic.

Subscriber

Registers interest in a named topic and executes a callback when compatible data arrives.

Many-to-Many Possibility

One publisher may serve several subscribers, and several publishers may write compatible messages to the same topic. System design must make this intentional.

Messages: Typed Data Contracts

Every topic carries a defined data structure

Intuition

A message type is a standardized form. It defines the fields and field types that communicating components agree to exchange.

Examples

A velocity command may contain linear and angular components. An image message includes pixel data and metadata. PX4 odometry includes estimated position, velocity, orientation, and timestamps.

Why Types Matter

A subscriber cannot safely interpret arbitrary bytes. Both sides must agree on the interface, units, coordinate conventions, and semantic meaning.

Live Tutorial: Inspect Topics

Discover names, types, rates, and current data

```
ros2 topic list
ros2 topic list -t
ros2 topic info /turtle1/cmd_vel --verbose
ros2 topic type /turtle1/cmd_vel
ros2 interface show geometry_msgs/msg/Twist
ros2 topic echo /turtle1/pose
ros2 topic hz /turtle1/pose
```

Observation Sequence

Find the topic, identify its type, inspect the interface fields, observe actual messages, and measure publication frequency.

Stop Streaming Commands

`ros2 topic echo` and `ros2 topic hz` continue until interrupted. Stop them with `Ctrl+C`.

Discover the Available Topics

Which communication channels currently exist?

List the Active Topics

```
ros2 topic list
```

Intuitive Interpretation

Each topic is a named communication channel:

- ▶ `/turtle1/cmd_vel`: receives motion commands;
- ▶ `/turtle1/pose`: publishes the turtle state;
- ▶ `/turtle1/color_sensor`: publishes sensed background color.

Central Idea

`ros2 topic list` shows which communication channels exist, but it does not show their message types or current data.

Discover Topics and Their Types

What kind of message travels through each channel?

Execute

```
ros2 topic list -t
```

How to Read One Line

```
/turtle1/cmd_vel  →  geometry_msgs/msg/Twist
```

The first element is the **topic name**. The value in brackets is the **message type**.

Central Idea

A topic is the channel. The message type defines the structure of the information allowed to travel through that channel.

Inspect One Topic

Who is connected to the velocity-command channel?

1. Execute the Command

```
ros2 topic info /turtle1/cmd_vel --verbose
```

2. The Terminal Output

Type: geometry_msgs/msg/Twist

Publisher count: 0

Subscription count: 1

...

4. The Engineering Conclusion

The turtle is fully awake and ready to receive velocity commands, but no controller program is currently publishing them. **This explains exactly why the turtle is stationary on your screen.**

3. Intuitive Interpretation

Let's break down exactly what the system is telling us:

- ▶ **Type:** The expected message structure is Twist (linear & angular velocity).
- ▶ **Sub count = 1:** One node (turtlesim) is actively listening and waiting for commands.
- ▶ **Pub count = 0:** Absolutely no node is currently sending commands into this topic.

Inspect the Message Contract

What exact information must a velocity command contain?

1. Find the Message Type

```
$ ros2 topic type /turtle1/cmd_vel
geometry_msgs/msg/Twist
```


Inspect the Message Contract

What exact information must a velocity command contain?

1. Find the Message Type

```
$ ros2 topic type /turtle1/cmd_vel  
geometry_msgs/msg/Twist
```

3. The Engineering Contract

A `Twist` message is the universal ROS 2 standard for rigid body movement. It enforces two 3D vectors:

- ▶ **linear:** Translational velocity (meters/second).
- ▶ **angular:** Rotational velocity (radians/second).

4. Application to the Turtle (and Drone)

Because the turtle lives in a flat 2D world, it only needs two of these six variables:

- ▶ `linear.x` to drive forward/backward.
- ▶ `angular.z` to spin left/right.

Observe Live Topic Data

What is the turtle publishing right now?

1. Execute the Echo Command

Listen to the live stream of data:

```
$ ros2 topic echo /turtle1/pose
```

2. The Continuous Output

```
x: 5.544445 y: 5.544445  
theta: 0.0 linear_velocity: 0.0  
angular_velocity: 0.0
```

4. The Drone Connection

Right now, both velocities are zero (the turtle is stationary). **Later this week, you will use this exact same command to read the live GPS and IMU data streaming from your 3D drone in Gazebo!**

3. Breaking Down the Telemetry

- ▶ **x and y:** Current 2D position in the grid.
- ▶ **theta:** Current orientation (yaw).
- ▶ **velocities:** Current physical speed.
- ▶ **--:** Separates messages. Because this data publishes continuously (e.g., at 62 Hz), the terminal will stream endlessly until you press Ctrl+C.

Measure the Publication Frequency

How frequently does the pose topic deliver new messages?

1. Execute the Hertz Command

Measure the data rate in real-time:

```
$ ros2 topic hz /turtle1/pose
```

2. The Terminal Output

```
average rate: 62.504
  min: 0.015s
  max: 0.017s
std dev: 0.00047s
window: 64
```

3. The Math & Interpretation

An average rate of ≈ 62.5 Hz means the turtle broadcasts its position 62.5 times per second.

The time between two messages (the period) is:

$$T = \frac{1}{f} = \frac{1}{62.5} \approx 0.016 \text{ seconds}$$

Meaning: Your flight controller will have exactly **16 milliseconds** to process this data and react!

4. The Engineering Caveat

A high frequency confirms the sensor is talking **fast**, but it does not prove the data is **accurate**. A broken drone GPS can perfectly publish the wrong coordinates at 100 Hz!

Moving from discovery to quantitative verification

<code>topic list</code>	Discover channels
<code>topic info</code>	Check connections
<code>topic type</code>	Find message name
<code>interface show</code>	Inspect data fields
<code>topic echo</code>	View live values
<code>topic hz</code>	Measure frequency

Complete Topic Inspection Workflow

Moving from discovery to quantitative verification

1. The ROS 2 Cheat Sheet

<code>topic list</code>	Discover channels
<code>topic info</code>	Check connections
<code>topic type</code>	Find message name
<code>interface show</code>	Inspect data fields
<code>topic echo</code>	View live values
<code>topic hz</code>	Measure frequency

2. The Debugging Sequence

When a robotics system fails, ask in order:

1. **Does the channel exist?** (`list`)
2. **Is anyone listening?** (`info`)
3. **What is the contract?** (`show`)
4. **Is data actually flowing?** (`echo`)
5. **Do the numbers make sense?** (`echo`)
6. **Is it fast enough to fly?** (`hz`)

Complete Topic Inspection Workflow

Moving from discovery to quantitative verification

1. The ROS 2 Cheat Sheet

<code>topic list</code>	Discover channels
<code>topic info</code>	Check connections
<code>topic type</code>	Find message name
<code>interface show</code>	Inspect data fields
<code>topic echo</code>	View live values
<code>topic hz</code>	Measure frequency

2. The Debugging Sequence

When a robotics system fails, ask in order:

1. **Does the channel exist?** (`list`)
2. **Is anyone listening?** (`info`)
3. **What is the contract?** (`show`)
4. **Is data actually flowing?** (`echo`)
5. **Do the numbers make sense?** (`echo`)
6. **Is it fast enough to fly?** (`hz`)

3. The Project Connection

Memorize this 6-step loop. Whether your turtle refuses to move today, or your **PX4 drone** refuses to accept YOLOv8 object-detection commands next week, this is the exact workflow you will use to find the bug.

Live Tutorial: Publish a Topic Message

Send a typed velocity command from the terminal

```
ros2 topic pub --once /turtle1/cmd_vel \  
geometry_msgs/msg/Twist \  
{linear: {x: 1.0}, angular: {z: 0.5}}"
```

What Happens

The command-line tool temporarily acts as a publisher. It creates one Twist message and sends it to the turtle's velocity command topic.

Flight-Safety Boundary

Terminal publication is appropriate for controlled tutorials and simulation. Never publish experimental commands to a real vehicle without an approved safety process and operator control.

Quality of Service

How should ROS 2 deliver the messages?

Simple Intuition

QoS is the **delivery agreement** between a publisher and a subscriber.

It answers a practical question:

Should we prioritize receiving every message, or receiving the newest message quickly?

Quality of Service

How should ROS 2 deliver the messages?

Simple Intuition

QoS is the **delivery agreement** between a publisher and a subscriber.

It answers a practical question:

Should we prioritize receiving every message, or receiving the newest message quickly?

Live Sensor Data

Camera images and drone positions arrive continuously.

If one sample is lost, the next sample will arrive soon. Receiving the **latest information** is often more useful than waiting for an old sample.

Fresh data first

Important Information

Configuration or mission-state messages may arrive less often.

Missing one of these messages may leave a component with incomplete or outdated information.

Reliable delivery first

Quality of Service

How should ROS 2 deliver the messages?

Important Compatibility Rule

A publisher and subscriber must use compatible QoS settings.

They may use the same topic name and message type, but still exchange no data if their delivery agreements are incompatible.

Three QoS Questions

1. How important is delivery?

Reliable tries to deliver every message.

Best effort sends messages without repeatedly recovering every missed sample.

2. How many messages should we keep?

The queue depth defines how many recent messages may wait for processing.

3. Do the publisher and subscriber agree?

Their QoS settings must be compatible before communication can occur.

Three QoS Questions

1. How important is delivery?

Reliable tries to deliver every message.

Best effort sends messages without repeatedly recovering every missed sample.

2. How many messages should we keep?

The queue depth defines how many recent messages may wait for processing.

3. Do the publisher and subscriber agree?

Their QoS settings must be compatible before communication can occur.

Drone Example

For high-rate odometry, the newest position is normally more useful than a long queue of old positions. A delayed history could make the software react to where the drone **was**, rather than where it **is now**.

QoS controls how data is delivered, not what the data means.

Check the communication agreement

```
ros2 topic info /topic_name --verbose
```

- ▶ Is there at least one publisher?
- ▶ Is there at least one subscriber?
- ▶ Do both use the expected message type?
- ▶ Are their QoS policies compatible?

Seeing the topic name does not prove that messages are being exchanged.
First verify the endpoints, message type, QoS settings, and actual data flow.

Mini-Project: Control and Inspect a ROS 2 System

Copy, execute, observe, and explain

Control a simulated turtle using only ROS 2 terminal commands

Command

Publish velocity commands from the terminal.

Observe

Watch the turtle and inspect its pose.

Understand

Identify nodes, topics, messages, and QoS.

Mini-Project: Control and Inspect a ROS 2 System

Copy, execute, observe, and explain

Control a simulated turtle using only ROS 2 terminal commands

Command

Publish velocity commands from the terminal.

Observe

Watch the turtle and inspect its pose.

Understand

Identify nodes, topics, messages, and QoS.

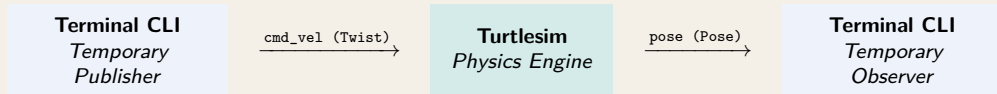
Learning Method

For every command:

1. predict what should happen;
2. execute the command;
3. observe the result;
4. explain the result using ROS 2 vocabulary.

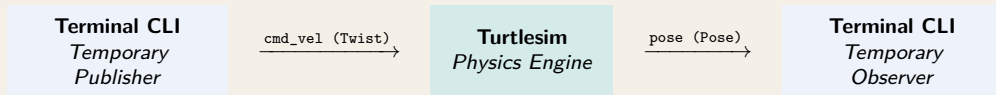
System Model

Three ROS 2 participants connected through decoupled topics



System Model

Three ROS 2 participants connected through decoupled topics



What Students Should Understand

- ▶ **Actuate:** Typing `ros2 topic pub` literally creates a temporary ROS 2 node behind the scenes.
- ▶ **Simulate:** Turtlesim subscribes to the velocity and physically moves the turtle.
- ▶ **Observe:** Typing `ros2 topic echo` creates a subscriber to read the new pose.

The Central Idea: Decoupling

The terminal doesn't know the turtle exists; it only knows the *topic* exists.

Topics carry messages between participants seamlessly, eliminating the need to write one massive, monolithic program.

Step 1: Start the Simulator

Terminal 1 creates the turtlesim node

Copy into Terminal 1

```
source /opt/ros/humble/setup.bash  
  
ros2 run turtlesim turtlesim_node
```

Expected Observation

A turtlesim window opens with one turtle near the center.

The terminal also displays information from the running `/turtlesim` node.

Interpretation

- ▶ `ros2 run` starts an executable from a ROS 2 package.
- ▶ `turtlesim` is the package.
- ▶ `turtlesim_node` is the executable.

Step 2: Discover the ROS 2 System

Terminal 2 identifies the active nodes and topics

Copy into Terminal 2

```
source /opt/ros/humble/setup.bash  
ros2 node list  
ros2 topic list -t
```

Step 2: Discover the ROS 2 System

Terminal 2 identifies the active nodes and topics

Copy into Terminal 2

```
source /opt/ros/humble/setup.bash
ros2 node list
ros2 topic list -t
```

Expected Observations

The node list should include:

```
/turtlesim
```

The topic list should include:

```
/turtle1/cmdvel [geometry_msgs/msg/Twist]
/turtle1/pose [turtlesim/msg/Pose]
```

Step 2: Discover the ROS 2 System

Terminal 2 identifies the active nodes and topics

Copy into Terminal 2

```
source /opt/ros/humble/setup.bash
ros2 node list
ros2 topic list -t
```

Expected Observations

The node list should include:

/turtlesim

The topic list should include:

/turtle1/cmdvel [geometry_msgs/msg/Twist]
/turtle1/pose [turtlesim/msg/Pose]

Explain the Result

/turtle1/cmd_vel carries motion commands.

/turtle1/pose carries the measured turtle state.

Step 3: Inspect the Command Topic

Who is waiting for velocity commands?

Copy into Terminal 2

```
ros2 topic info /turtle1/cmd_vel --verbose
```

Expected Observation Before Publishing

Look for information similar to:

Type: geometry_msgs/msg/Twist

Publisher count: 0

Subscription count: 1

Interpretation

- ▶ The topic expects a Twist message.
- ▶ Turtlesim is subscribed and waiting for commands.
- ▶ No publisher is currently sending commands.
- ▶ Therefore, the turtle remains stationary.

Prediction

What should happen to the publisher count when we start publishing from another terminal?

Step 4: Inspect the Message Structure

Understand the command before publishing it

Copy into Terminal 2

```
ros2 interface show geometry_msgs/msg/Twist
```

Important Fields

Vector3 linear

float64 x

float64 y

float64 z

Vector3 angular

float64 x

float64 y

float64 z

linear.x

Controls the turtle's forward or backward velocity.

angular.z

Controls the turtle's rotational velocity.

Prediction

Forward motion combined with continuous rotation should produce an approximately circular trajectory.

Step 5: Publish a Circular-Motion Command

Terminal 3 becomes a temporary ROS 2 publisher

Copy into Terminal 3

```
source /opt/ros/humble/setup.bash

ros2 topic pub --rate 10 \
  /turtle1/cmd_vel \
  geometry_msgs/msg/Twist \
  "{linear: {x: 1.5}, angular: {z: 0.8}}"
```

Expected Observation

The turtle should begin moving along an approximately circular path.

The terminal repeatedly publishes the same velocity command at approximately 10 messages per second.

Keep Terminal 3 Running

Do not press Ctrl+C until the following inspection steps have been completed.

Step 7: Observe One Command Message

Inspect the information travelling through the topic

Copy into Terminal 2

```
ros2 topic echo /turtle1/cmd_vel --once
```

Expected Message

```
linear:  
  x: 1.5  
  y: 0.0  
  z: 0.0  
angular:  
  x: 0.0  
  y: 0.0  
  z: 0.8
```

Interpretation

- ▶ `linear.x = 1.5`: move forward.
- ▶ `angular.z = 0.8`: rotate continuously.
- ▶ Unspecified fields use zero in this message.

Important Distinction

The topic is the communication channel. The Twist message is the structured information travelling through it.

Step 8: Measure the Command Frequency

Verify how often the publisher sends messages

Copy into Terminal 2

```
ros2 topic hz /turtle1/cmd_vel
```

Expected Observation

The average rate should be close to:

average rate: 10.0

The exact value may vary slightly.

Interpretation

A frequency of 10 Hz means that the publisher sends approximately ten velocity messages every second.

$$T = \frac{1}{f} = \frac{1}{10} = 0.1 \text{ s}$$

Stop the Measurement

Press **Ctrl+C** in Terminal 2 to stop only the frequency measurement. The publisher in Terminal 3 continues running.

Step 9: Observe the Measured Turtle Pose

Terminal 4 becomes a subscriber to the pose topic

Copy into Terminal 4

```
source /opt/ros/humble/setup.bash  
  
ros2 topic echo /turtle1/pose
```

Observe the Changing Fields

```
x: ...  
y: ...  
theta: ...  
linear_velocity: ...  
angular_velocity: ...
```

Interpretation

- ▶ x and y: current position;
- ▶ theta: current orientation;
- ▶ linear_velocity: measured forward velocity;
- ▶ angular_velocity: measured rotational velocity.

Command Versus Measurement

/turtle1/cmd_vel carries the requested motion.

/turtle1/pose carries the state reported by the simulator.

Step 10: Inspect the Pose Topic

Identify the original publisher and the temporary subscriber

While Terminal 4 Is Still Echoing,
Copy into Terminal 2

```
ros2 topic info /turtle1/pose --verbose
```

Expected Interpretation

- ▶ Turtlesim is the publisher of the pose.
- ▶ The `ros2 topic echo` command is a temporary subscriber.
- ▶ The message type is `turtlesim/msg/Pose`.
- ▶ The verbose output displays the endpoint QoS profiles.

Important Observation

The ROS 2 command-line tools are not passive text viewers. Commands such as `topic echo` temporarily create real ROS 2 participants in the communication graph.

Step 11: Inspect the QoS Profile

Understand the communication rules used by each endpoint

Use the Existing Verbose Output

Look for fields such as:

Reliability: RELIABLE

History: KEEP_LAST

Depth: ...

Durability: VOLATILE

Mailbox Analogy

QoS defines the delivery rules of the mailbox: how delivery is attempted, how many messages may wait, and whether earlier messages are preserved.

Reliability and History

RELIABLE requests dependable delivery.

KEEP_LAST preserves only a limited number of recent messages.

Depth and Durability

Depth defines the queue size.

VOLATILE means old messages are not saved for subscribers that connect later.

Step 12: Stop the Command Publisher

Observe what changes when one participant disappears

In Terminal 3

Press:

Ctrl+C

Expected Observation

The repeated velocity publisher stops.

The turtle should stop receiving new velocity commands.

Then Copy into Terminal 2

```
ros2 topic info /turtle1/cmd_vel --verbose
```

Expected Graph Change

The publisher count should return to zero, while turtlesim remains subscribed and ready to receive future commands.

Step 13: Publish One Stop Command

Send a single zero-velocity message

Copy into Terminal 3

```
ros2 topic pub --once \  
  /turtle1/cmd_vel \  
  geometry_msgs/msg/Twist \  
  "{linear: {x: 0.0}, angular: {z: 0.0}}"
```

Expected Observation

The command publishes exactly one Twist message containing zero forward and rotational velocity.

Why Use -once?

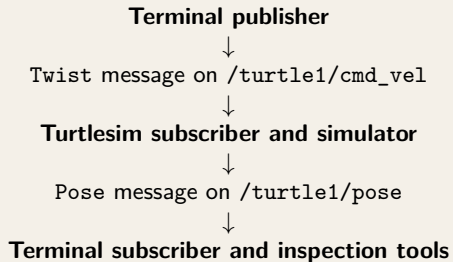
The objective is to send one specific command and then allow the temporary publisher to terminate.

Project Connection

The same distinction appears in robotics systems: some publishers send continuous setpoints, while others publish only when new information is available.

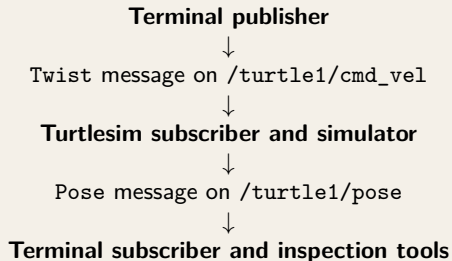
Mini-Project Summary

From a terminal command to an observable ROS 2 system



Mini-Project Summary

From a terminal command to an observable ROS 2 system



Expected Mental Model

A node performs a focused responsibility, a topic provides a named communication channel, a message defines the transmitted data, and QoS defines how that communication behaves.

1 ROS 2 Humble Foundations

- Environment and First Observation
- The ROS 2 Graph and Nodes
- Topics, Publishers, Subscribers, and Messages
- **Services and Actions**
- Parameters, Packages, and Workspaces
- Python Publisher and Subscriber
- Drone-System Integration
- Troubleshooting and Practice

Services: Request and Response

Ask for one operation and receive one result

Client request → **Service server** → **response**

Intuition

A service resembles asking a technician one question and waiting for one answer. It is best for short operations that complete quickly.

Drone Examples

Start an inspection mission, reset a component, query one configuration state, or request a controlled operation with an immediate response.

Not for Continuous Data

Use topics for streams. Do not repeatedly call a service to imitate high-rate telemetry.

Live Tutorial: Discover and Call a Service

Create a second turtle through a request-response interaction

```
ros2 service list
ros2 service type /spawn
ros2 interface show turtlesim/srv/Spawn
ros2 service call /spawn turtlesim/srv/Spawn \
"{x: 2.0, y: 2.0, theta: 0.0, name: 'inspector'}"
```

Read the Interaction

The service type defines request fields and response fields. The client sends position, orientation, and name; the server returns the created turtle name.

Common Mistake

A correct service name with an incorrect service type or malformed field structure will fail. Inspect the interface before composing the request.

Actions: Long-Running Goals

A goal may need feedback, time, completion, or cancellation

Goal $\longrightarrow$ repeated feedback $\longrightarrow$ result
with the possibility of cancellation

Intuition

An action resembles assigning a technician a task that takes time. You receive progress updates, can cancel if needed, and receive a final result.

Drone Examples

Execute a planned inspection segment, navigate to a waypoint, or perform a long-running scan where progress and cancellation matter.

Live Tutorial: Inspect and Send an Action Goal

Rotate the turtle while observing feedback

```
ros2 action list -t
ros2 action info /turtle1/rotate_absolute
ros2 interface show turtlesim/action/RotateAbsolute
ros2 action send_goal --feedback \
/turtle1/rotate_absolute \
turtlesim/action/RotateAbsolute "{theta: 1.57}"
```

What to Observe

The action client sends a target angle, receives periodic feedback while rotation continues, and finally receives a result.

Concept Check

If an operation is immediate, a service may fit. If it takes time and needs feedback or cancellation, an action is usually the clearer model.

Topics, Services, or Actions?

Choose the interface from the communication need

- ▶ **Topic:** repeated stream with no direct response, such as odometry or images.
- ▶ **Service:** short request with one response, such as resetting or starting a simple operation.
- ▶ **Action:** long-running goal with feedback, result, and cancellation, such as executing a mission segment.

Design Question

Do not begin by asking, “Which ROS 2 feature do I want to use?” Ask, “What communication behavior does this engineering interaction require?”

1 ROS 2 Humble Foundations

- Environment and First Observation
- The ROS 2 Graph and Nodes
- Topics, Publishers, Subscribers, and Messages
- Services and Actions
- Parameters, Packages, and Workspaces
- Python Publisher and Subscriber
- Drone-System Integration
- Troubleshooting and Practice

Parameters: Node Configuration

Adjust behavior without rewriting source code

Intuition

Parameters are named configuration values associated with a node. They may express limits, rates, target names, radii, durations, or feature choices.

Drone Example

The orbit controller may expose `radius_m`, `altitude_m`, `orbit_period_s`, and `face_center` as parameters.

Validation

A configurable value still requires type, range, unit, and consistency checks before it influences vehicle behavior.

Live Tutorial: Inspect Parameters

Discover and modify node configuration

```
ros2 param list /turtlesim
ros2 param describe /turtlesim background_r
ros2 param get /turtlesim background_r
ros2 param set /turtlesim background_r 80
ros2 service call /clear std_srvs/srv/Empty "{}"
```

Observation

The parameter value changes the background configuration. Calling /clear redraws the window so the new value becomes visible.

Project Transfer

Use `ros2 param list /orbit_controller` to discover mission parameters instead of searching blindly through source code.

Packages: Deliverable Units of ROS 2 Software

Group code, metadata, configuration, and launch resources

Definition

A ROS 2 package is an organizational and build unit containing related executables, libraries, interfaces, tests, configuration, launch files, and dependency metadata.

Drone Examples

A mission package may contain the orbit controller and trajectory utilities. A perception package may contain camera-processing nodes and detector configuration.

Package Responsibility

A package should communicate a coherent purpose. Do not place every internship file into one undefined package.

Live Tutorial: Inspect Installed Packages

Use the package index rather than guessing locations

```
ros2 pkg list | grep turtlesim  
ros2 pkg executables turtlesim  
ros2 pkg prefix turtlesim  
ros2 interface package turtlesim
```

Interpretation

These commands confirm that the package is discoverable, list its executables, identify its installation prefix, and display its interfaces.

Environment Reminder

If a newly built package is missing, verify that the correct workspace was built and that its `install/setup.bash` file was sourced in the current terminal.

Live Tutorial: Create a Humble Python Workspace

Build in a separate practice area before modifying the internship packages

```
source /opt/ros/humble/setup.bash
mkdir -p ~/ros2_humble_practice/src
cd ~/ros2_humble_practice/src

ros2 pkg create --build-type ament_python \
--license Apache-2.0 \
--node-name telemetry_demo robotics_practice
```

What Is Created

A Python ROS 2 package containing metadata, Python package structure, resource index entry, setup files, and a starter node.

Naming

Use lowercase package names with underscores. Do not create the practice package inside the installed ROS directory or an unrelated Git repository.

Live Tutorial: Build and Source the Workspace

Source code becomes a discoverable ROS 2 package

```
cd ~/ros2_humble_practice
colcon build --symlink-install
source install/setup.bash
```

```
ros2 pkg executables robotics_practice
ros2 run robotics_practice telemetry_demo
```

Why `-symlink-install`?

For Python development, editable source files are linked into the install space, reducing unnecessary rebuilds for many code-only changes. Metadata changes may still require rebuilding.

Overlay Order

Source the base Humble installation first, then the workspace overlay. A new terminal forgets both until they are sourced again.

1 ROS 2 Humble Foundations

- Environment and First Observation
- The ROS 2 Graph and Nodes
- Topics, Publishers, Subscribers, and Messages
- Services and Actions
- Parameters, Packages, and Workspaces
- **Python Publisher and Subscriber**
- Drone-System Integration
- Troubleshooting and Practice

Python Publisher: The Communication Pattern

A timer creates and publishes typed telemetry

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32

class AltitudePublisher(Node):
    def __init__(self):
        super().__init__('altitude_publisher')
        self.publisher = self.create_publisher(
            Float32, '/demo/altitude_m', 10)
        self.timer = self.create_timer(1.0, self.publish_altitude)

    def publish_altitude(self):
        msg = Float32()
        msg.data = 4.0
        self.publisher.publish(msg)
```

Read It as a Story

Create a node, create a typed publisher, create a periodic timer, build one message, and publish it on a named topic.

Incoming telemetry triggers a focused method

Read It as a Story

U M 6 P

Live Tutorial: Verify a Publisher Before Integration

Use terminal tools as an independent observer

```
ros2 node list
ros2 topic list -t
ros2 topic info /demo/altitude_m --verbose
ros2 topic echo /demo/altitude_m
ros2 topic hz /demo/altitude_m
```

Layer-by-Layer Evidence

First confirm the node exists, then the topic and type, then endpoint compatibility, then actual values, and finally the observed rate.

Debugging Principle

Do not connect five unverified components and then ask why the pipeline fails. Verify each interface independently before integration.

1 ROS 2 Humble Foundations

- Environment and First Observation
- The ROS 2 Graph and Nodes
- Topics, Publishers, Subscribers, and Messages
- Services and Actions
- Parameters, Packages, and Workspaces
- Python Publisher and Subscriber
- Drone-System Integration
- Troubleshooting and Practice

Live Tutorial: Inspect the Drone Graph

Use the same discovery method on the full project

```
source /opt/ros/humble/setup.bash
cd ~/quard_um6p_intership
source config/project.env
# After building the workspace:
source install/setup.bash
```

```
ros2 node list
ros2 topic list -t
ros2 service list -t
ros2 action list -t
ros2 param list
```

Question for Every Interface

Who produces it? Who consumes it? What type does it use? At what rate? In which units and coordinate frame? What observation proves it works?

Live Tutorial: Inspect PX4 Interfaces

Understand direction before publishing or subscribing

```
ros2 topic list | grep /fmu
ros2 topic info /fmu/out/vehicle_odometry --verbose
ros2 interface show px4_msgs/msg/VehicleOdometry
ros2 topic echo /fmu/out/vehicle_odometry --once

ros2 topic info /fmu/in/trajectory_setpoint --verbose
ros2 interface show px4_msgs/msg/TrajectorySetpoint
```

Naming Intuition

For PX4 interfaces, /fmu/out/... carries data leaving PX4, while /fmu/in/... carries data entering PX4.

No Blind Publication

Inspecting a setpoint interface is safe. Publishing arbitrary flight setpoints is a separate controlled activity performed only in the documented simulation workflow.

Visualize the Graph with `rqt_graph`

A diagram complements terminal evidence

Execute While Nodes Are Running

```
rqt_graph
```

What to Trace

Locate the mission node, PX4 communication path, camera bridge, perception node, and topics connecting them. Compare the graph with the intended architecture.

Interpretation

A visible connection shows discovered ROS 2 endpoints. It does not prove correct values, rates, units, QoS, or system behavior. Use CLI inspection and observed outputs as additional evidence.

Record and Replay Topic Data

Preserve communication evidence for repeatable analysis

```
ros2 bag record /turtle1/pose
# Move the turtle, then stop recording with Ctrl+C.
ros2 bag info rosbag2_*
ros2 bag play rosbag2_*
```

Intuition

A ROS bag records serialized topic messages with timing information. It allows later inspection or replay without reproducing the original motion immediately.

Data Management

High-rate images and telemetry can create large files quickly. Record only required topics, monitor storage, and keep generated bag data out of normal Git history unless the project explicitly requires a small fixture.

1 ROS 2 Humble Foundations

- Environment and First Observation
- The ROS 2 Graph and Nodes
- Topics, Publishers, Subscribers, and Messages
- Services and Actions
- Parameters, Packages, and Workspaces
- Python Publisher and Subscriber
- Drone-System Integration
- Troubleshooting and Practice

A ROS 2 Troubleshooting Ladder

Move from environment to behavior without skipping layers

1. **Environment:** Was Humble sourced in this terminal?
2. **Process:** Is the expected executable still running?
3. **Node:** Does it appear in `ros2 node list`?
4. **Interface:** Does the topic, service, or action exist with the expected type?
5. **Compatibility:** Do endpoints and QoS policies match?
6. **Data:** Are values, rate, timestamp, units, and frame correct?
7. **Behavior:** Does the complete component produce the intended result?

Method

Change one factor at a time and preserve the original error. Random restarts may remove the symptom without explaining the cause.

Common Humble Problems and First Checks

Use observable evidence before reinstalling software

```
# ros2: command not found
source /opt/ros/humble/setup.bash

# Package not found after building
source ~/your_workspace/install/setup.bash

# Node expected but absent
ps aux | grep process_name
ros2 node list

# Topic exists but no data
ros2 topic info /topic_name --verbose
ros2 topic echo /topic_name

# Broad environment report
ros2 doctor --report
```

Do Not Guess

Confirm the Ubuntu release, ROS distribution, current workspace, active process, interface type, and complete terminal output before changing the installation.

Guided Exercise: Communication Detective

Explain a running ROS 2 system without reading its source code first

Mission

With Turtlesim running, identify both nodes, all velocity-related topics, the command message type, one service, one action, and one configurable parameter.

Required Evidence

Record the commands used, selected output, a simple hand-drawn graph, and one sentence explaining why each interface is a topic, service, action, or parameter.

Learning Goal

The objective is not command memorization. It is the ability to reconstruct the software architecture from observable ROS 2 interfaces.

Guided Exercise: Drone Interface Map

Transfer the same reasoning to the internship repository

Create a Small Interface Table in Your Notes

For one mission node and one perception node, record the node name, input interfaces, output interfaces, message types, expected rates, units, coordinate frames, and verification commands.

Minimum Questions

What information enters the component? What does it produce? Which ROS 2 mechanism is used? Who is connected? What evidence confirms meaningful data rather than only discovery?

Do Not Control the Vehicle Yet

This exercise is architecture inspection. Flight-command publication belongs to the later documented PX4 simulation tutorial.

ROS 2 Readiness Check

Students should now be able to explain and demonstrate

Conceptual Understanding

- ▶ ROS 2 versus Ubuntu
- ▶ graph, nodes, and interfaces
- ▶ publishers, subscribers, and messages
- ▶ topics, services, and actions
- ▶ packages, workspaces, and parameters

Practical Capability

- ▶ source ROS 2 Humble
- ▶ run and inspect nodes
- ▶ inspect and publish topic data
- ▶ call a service and send an action goal
- ▶ create, build, source, and run a package
- ▶ diagnose a missing interface systematically

Learning Roadmap

From communication foundations to the complete autonomous-drone system

ROS 2 graph → Python nodes → PX4 interfaces → Gazebo sensors → AI perception
→ mission evidence

Next Technical Step

Use these communication concepts to understand the project's Python nodes, PX4 message interfaces, image bridge, and launch sequence before integrating the full simulation pipeline.

Final Principle

A connected graph is only the beginning. A professional engineer also verifies interface types, QoS, data values, units, coordinate frames, timing, system behavior, and documented limitations.

Official ROS 2 Humble Resources I

Follow the Humble documentation for version-compatible tutorials



Open Robotics, *ROS 2 Humble Documentation*.

<https://docs.ros.org/en/humble/>



Open Robotics, *ROS 2 Humble Tutorials*.

<https://docs.ros.org/en/humble/Tutorials.html>



Open Robotics, *Beginner CLI Tools*.

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html>



Open Robotics, *ROS 2 Concepts*.

<https://docs.ros.org/en/humble/Concepts.html>



Francisco Martín Rico, *A Concise Introduction to Robot Programming with ROS2*.

Official ROS 2 Humble Resources II

Packages, Python nodes, and integration practice



Open Robotics, *Creating a Workspace*.

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html>



Open Robotics, *Creating a Package*.

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>



Open Robotics, *Writing a Simple Publisher and Subscriber in Python*.

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>



Open Robotics, *Writing a Simple Service and Client in Python*.

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Service-And-Client.html>